

JAVA DOCS

The Java Stream API

Declarative, Functional-Style Processing of Data Sequences

Java Agentic AI — Knowledge Base Reference

1. What Is a Stream?

A Stream is a sequence of elements supporting functional-style, declarative operations — filter, map, reduce, sorted, etc. Streams are NOT data structures; they don't store elements, they pull them on demand from a source (a collection, array, generator function, or I/O channel) and are single-use (consumed once, then unusable).

Property	Description
Lazy	Intermediate operations don't execute until a terminal operation is invoked
Single-use	A stream is consumed once; reusing a spent stream throws IllegalStateException
Non-mutating	Stream operations produce a new stream/result rather than modifying the source
Can be parallel	stream() runs sequentially, parallelStream() splits work across threads

2. Creating Streams

```
List<String> list = List.of("a", "b", "c");
Stream<String> s1 = list.stream();
Stream<String> s2 = Stream.of("x", "y", "z");
IntStream s3 = IntStream.range(0, 10);           // 0..9
IntStream s4 = IntStream.rangeClosed(1, 10);     // 1..10
Stream<Integer> s5 = Stream.iterate(1, n -> n * 2).limit(5); // 1,2,4,8,16
Stream<Double> s6 = Stream.generate(Math::random).limit(3);
Stream<String> s7 = Arrays.stream(new String[]{"p", "q"});
```

3. Intermediate Operations (Lazy — return a Stream)

Operation	Purpose
filter(Predicate)	Keep elements matching a condition
map(Function)	Transform each element 1-to-1
flatMap(Function)	Transform each element into a stream, then flatten all into one
sorted() / sorted(Comparator)	Sort elements (natural or custom order)
distinct()	Remove duplicates (uses equals())
limit(n) / skip(n)	Truncate to first n / skip first n elements
peek(Consumer)	Debug/inspect elements without altering the stream (avoid side effects in production)

```
List<String> words = List.of("banana", "apple", "cherry", "apple");

List<String> result = words.stream()
    .filter(w -> w.length() > 5)
    .map(String::toUpperCase)
    .distinct()
    .sorted()
    .collect(Collectors.toList());
```

```
// [BANANA, CHERRY]

List<Integer> lengths = List.of("hi", "world")
    .stream()
    .flatMap(s -> s.chars().boxed())    // flatten each String into its char codes
    .collect(Collectors.toList());
```

4. Terminal Operations (Eager — trigger execution, end the stream)

Operation	Returns
collect(Collector)	A collection, map, string, or custom accumulation
forEach(Consumer)	void — performs an action per element
reduce(identity, BinaryOperator)	A single accumulated value
count()	long — number of elements
anyMatch / allMatch / noneMatch(Predicate)	boolean
findFirst() / findAny()	Optional<T>
min(Comparator) / max(Comparator)	Optional<T>
toArray()	An array

```
int sum = List.of(1, 2, 3, 4).stream()
    .reduce(0, Integer::sum);           // 10

Optional<Integer> max = List.of(1, 5, 3).stream()
    .max(Integer::compareTo);

boolean hasNegative = List.of(1, -2, 3).stream().anyMatch(n -> n < 0); // true
```

5. Collectors — Building Results

```
List<String> names = List.of("Ann", "Bob", "Cara", "Dan");

List<String> asList = names.stream().collect(Collectors.toList());
Set<String> asSet = names.stream().collect(Collectors.toSet());
String joined = names.stream().collect(Collectors.joining(", ", "[", "]")); // [Ann, Bob, Cara, Dan]

Map<Integer, List<String>> byLength = names.stream()
    .collect(Collectors.groupingBy(String::length));
// {3=[Ann, Bob], 4=[Cara, Dan]}

Map<Boolean, List<String>> partitioned = names.stream()
    .collect(Collectors.partitioningBy(n -> n.length() > 3));

double avgLength = names.stream()
```

```

        .collect(Collectors.averagingInt(String::length));

Map<String, Integer> nameToLength = names.stream()
    .collect(Collectors.toMap(n -> n, String::length));

// Downstream collectors: count per group
Map<Integer, Long> countByLength = names.stream()
    .collect(Collectors.groupingBy(String::length, Collectors.counting()));

```

6. Primitive Streams — IntStream, LongStream, DoubleStream

Boxing every int into an Integer object for a Stream is wasteful. Primitive-specialized streams avoid autoboxing overhead and expose numeric aggregate operations directly.

```

IntStream.rangeClosed(1, 5).sum();           // 15
IntStream.rangeClosed(1, 5).average();       // OptionalDouble
IntSummaryStatistics stats = IntStream.rangeClosed(1, 100).summaryStatistics();
stats.getMax(); stats.getMin(); stats.getAverage(); stats.getCount();

List<Integer> boxed = IntStream.rangeClosed(1, 5).boxed().collect(Collectors.toList());
int[] arr = List.of(1, 2, 3).stream().mapToInt(Integer::intValue).toArray();

```

7. Parallel Streams

```

long count = hugeList.parallelStream()
    .filter(x -> x > 0)
    .count();

```

Note: Parallel streams split work across the common ForkJoinPool (sized to the number of CPU cores by default) — beneficial for large datasets with CPU-bound, stateless, independent operations, but can perform WORSE than sequential for small datasets due to thread coordination overhead, and are dangerous with stateful/shared-mutable operations or non-associative reduce functions.

8. Stream Pipeline Execution Model

Streams execute lazily, element-by-element through the whole pipeline (vertically), not stage-by-stage across all elements (horizontally) — this allows short-circuiting operations like `findFirst()/anyMatch()/limit()` to stop processing early without touching every source element.

```

List.of(1, 2, 3, 4, 5).stream()
    .peek(n -> System.out.println("filter check: " + n))
    .filter(n -> n % 2 == 0)
    .peek(n -> System.out.println("map: " + n))
    .map(n -> n * n)
    .findFirst();

// Output shows element 1 fails filter, element 2 passes filter then gets mapped,
// then the pipeline stops immediately - element 3, 4, 5 are never even checked.

```

9. Quick Interview Recap

- Intermediate vs terminal operations? Intermediate ops (filter, map) are lazy and return a new stream; terminal ops (collect, forEach, reduce) trigger actual execution and end the stream.
- Can a stream be reused? No — once a terminal operation runs, the stream is consumed; reusing it throws `IllegalStateException`.
- `map()` vs `flatMap()`? `map()` does a 1-to-1 transform; `flatMap()` transforms each element into its own stream and flattens all of them into a single result stream (e.g. `List > -> Stream`).
- When should you avoid `parallelStream()`? For small datasets, I/O-bound tasks, stateful lambdas, or operations that aren't associative — the coordination overhead or incorrect results can outweigh any parallel speedup.
- How does short-circuiting work? Operations like `limit()`, `findFirst()`, and `anyMatch()` can stop pipeline execution before processing the entire source, because streams process one element through the whole pipeline at a time rather than stage-by-stage.